

Construirea unui modul de kernel in Linux

Obiectivul lucrării

Lucrarea își propune familiarizarea cu funcționalitatea unui driver precum și cu modul în care componente. Interacțiunea dintre procesele rulând în modul utilizator și cele rulând în cadrul kernelului Linux este prezentată și este exemplificată prin scrierea unui modul de kernel Linux.

Breviar teoretic

Kernelul unui sistem de operare este centrul unui sistem de operare și are control asupra funcționării componentelor sistemului atât hardware cât și software. Kernelul este unul dintre primele programe încărcate la pornirea sistemului de operare. Kernelul Linux este unul monolitic, adică întregul sistem de operare operează în spațiul privilegiat care intermediază interacțiunea între aplicațiile utilizatorilor și hardware.

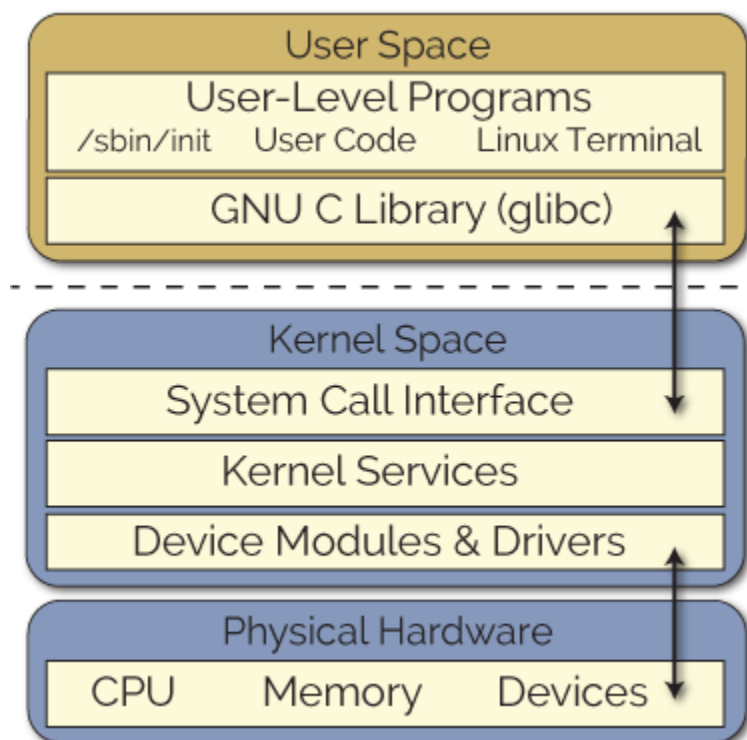


Fig. 1 Spațiul utilizator este cel în care rulează aplicațiile care au nevoie de kernel pentru a accesa componentele hardware

În spațiul kernel se găsesc drivere pentru dispozitive și modulele kernel. Un driver pentru dispozitive oferă o interfață software pentru funcțiile oferite de un dispozitiv comentat

la sistemul de calcul prin care se realizează controlul acestui dispozitiv. Procesele rulând în kernelul sistemului de operare au o viteză mare de execuție însă sunt cel mai puțin protejate în cazul unei execuții incorecte, afectând stabilitatea sistemului de operare.

Zonele de memorie pentru spațiul de utilizator și cel de kernel sunt separate. Serviciile din kernel care sunt puse la dispoziția aplicațiilor rulând în spațiul utilizator sunt controlate prin folosirea apelurilor de sistem. Apelurile de sistem pot implica controlul proceselor, manipularea fișierelor și a dispozitivelor, gestionarea informațiilor (de exemplu actualizarea orei) și a comunicațiilor între procese (Inter Process Communication).

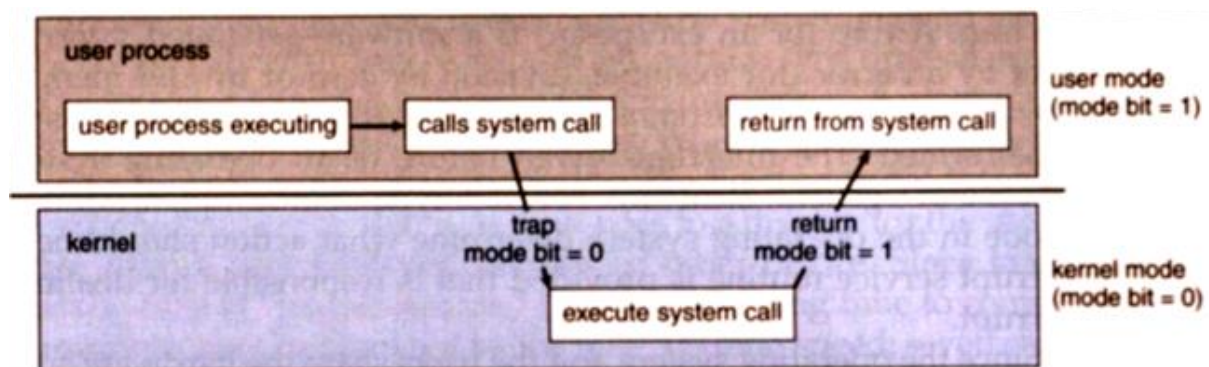


Fig. 2 Un System Call asigură o interfață cu serviciile oferite de sistemul de operare

Modulele kernel sunt secvențe de cod care pot fi să nu încarcate în kernel în funcție de cerințele utilizatorilor. Un driver poate fi legat static de kernelul sistemului de operare sau poate fi încărcat în mod dinamic drept un modul kernel. Încărcarea statică implică compilarea în kernelul sistemului de operare a secvenței de cod a driverului și poate fi motivată de necesitatea ca driverul să fie prezent la momentul pornirii sistemului de calcul sau pentru a crește securitatea sistemului. Încărcarea dinamică a driverelor este însă tehnica recomandată.

Nu toate modulele de kernel sunt driveri, de exemplu este posibilă încărcarea drept modul kernel a unui diferit algoritm pentru planificarea proceselor care să înlocuiască algoritmi impliciti.

Fără această posibilitate de încărcare modulară, kernelul Linux ar deveni foarte mare deoarece ar trebui să suporte toate driverele existente și nu ar fi flexibil în cazul apariției de driveri noi, deoarece ar necesita o recompilare a kernelului (proces care nu este foarte simplu).

Structura de bază a unui program modul de kernel este următoarea:

/ Incluziunile sunt necesare pentru orice modul. Ele includ și definițiile funcțiilor macro module_init și module_exit pe care o vom folosi în continuare. */*

```
#include <linux/init.h>
#include <linux/module.h>
```

/ Aceasta este funcția de initializare a modulului care este executată la încărcarea acestuia. Cuvântul __init informează kernelul că acest cod va fi executat o singură dată. */*

```
static int functie_init(void)
{
    return 0;
```

```
    }  
/* Aceasta este functia care se executa la scoaterea modulului. Cuvântul __exit informează kernelul ca  
acest cod va fi executat o singura data. */
```

```
static void functie_exit(void)  
{  
    return 0;  
}
```

/ Cele doua functii precizează kernelului momentul in care funcțiile anterioare sunt executate, la
pornirea respectiv oprirea modulului. */*

```
module_init(functie_init);  
module_exit(functie_exit);
```

Se observa ca un modul kernel nu are funcția main(). Fiecare modul are o funcție de intrare (încărcare in kernel) si o funcție de ieșire. Modulul anterior nu face altceva decât sa se încarce si sa iasă din kernel. Kernelul este informat despre aceste operații prin funcțiile macro *module_init* si *module_exit*. Includerea fișierului header *module.h* este necesara pentru a avea informații despre versiunea de kernel pentru care modulul este proiectat. Totodată acest modul conține declarațiile pentru *module_init* si *module_exit*.

Aplicație

In continuare vom crea un modul kernel pe care îl vom compila si încarcă drept un modul in sistemul de operare. Pentru acest exemplu se va folosi Ubuntu Linux 16.04.

Dezvoltarea tradițională a unei aplicații nu mai poate fi aplicată dezvoltării unui modul de kernel. Codul scris trebuie sa răspundă la evenimentele sistemului de calcul in locul proiectării unei logici secvențiale de execuție. Suplimentar funcțiile folosite in limbajul C sunt asigurate de librăriile C, in vreme ce modulele de kernel utilizează doar funcțiile oferite de kernel.

Pentru a vedea diferența intre o funcție si un modul de kernel se va scrie programul:

```
int main(void)  
{ printf("Hello world"); return 0; }
```

După ce se va compila într-un fișier executabil (numit in acest caz *test*), se va executa împreuna cu funcția de monitorizare a execuției unui program, *strace*:

```
strace ./test
```

Pe ecran vor fi afișate toate apelurile de sistem pe care procesul le realizează pe durata execuției, deși programul executa decât instrucțiunea *printf*. Scrierea de module de kernel care înlocuiesc apelurile de sistem este des folosita in atacurile informatice pentru a introduce porți de acces intr-un sistem de operare de către persoane rău voitoare.

Pentru modulele de kernel doar câteva funcții sunt oferite precum **printk** (cu funcționalitate similara *printf*) sau **kmalloc** (cu funcționalitate similara *malloc*). Suplimentar eliberarea memoriei alocate este lăsată in sarcina programatorului deoarece nu există astfel de mecanisme implementate la acest nivel. Programarea se face in limbajele C si ASM deoarece librăriile C++ sunt mult prea mari pentru a fi stocate in kernel.

Pentru a rula exemplul de mai jos va folosi un utilizator obișnuit (creat de exemplu în timpul instalării sistemului de operare Ubuntu), care însă are drepturi administrative (poate executa comenzi sudo). Se poate lucra însă și drept utilizatorul root dacă se dorește.

Pentru început se vor actualiza sursele sistemului de operare:

apt-get update

În continuare se vor instala mai multe unele pentru dezvoltarea de module kernel:

apt-get install build-essential linux-headers-`uname -r`

Comanda **uname -r** identifică versiunea kernelului folosită de sistemul de operare. Dacă sunt probleme în execuția comenzii anterioare se va executa comanda separat și se va completa corespunzător după: *linux-headers-*

Apoi se va crea un folder în care vom lucra:

mkdir modul_kernel

cd modul_kernel

În continuare vom scrie în fișierul **hello.c** următoarea secvență de cod:

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>

/* Urmatoarele functii ofera kernelului informatii despre modul. In cazul in care aceste informatii nu
sunt precizate, un mesaj de atentionare va fi afisat la incarcarea modului in kernel: kernel tainted*/
MODULE_LICENSE("GPL");
MODULE_AUTHOR("Laborator T201");
MODULE_DESCRIPTION("Un modul kernel de test");
MODULE_VERSION("1");

/* Aceasta este functia de initializare a modulului care este executata la incarcarea acestuia. Functia
printk scrie in bufferul de mesaje al kernelului */
static int __init example_init(void) {
    printk(KERN_INFO "Incepere modul kernel!\n");
    return 0;
}

/* Aceasta este functia care se executa la scoaterea modulului. */
static void __exit example_exit(void) {
    printk(KERN_INFO "Terminare modul kernel!\n");
}
```

```
}
```

```
module_init(example_init);
```

```
module_exit(example_exit);
```

Deoarece comanda pentru compilarea modulului este foarte lunga, se recomanda scrierea acesteia într-un fișier Makefile care va executa instrucțiunile necesare. Pentru acest lucru se va crea un fișier numit Makefile in care se va scrie următorul conținut.

Atenție! Se vor utiliza editoarele nano sau vi deoarece diferitele comenzi din fișierul Makefile necesita la început un TAB si nu spațiu!

*Atenție! Daca sunt probleme in execuția fișierului Makefile, se va executa separat in linia de comanda **uname -r** si se va introduce rezultatul acestei comenzi in locul zonei marcate cu bold.*

```
obj-m += modul_kernel.o
all:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules
clean:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

Se executa fișierul makefile cu ajutorul instrucțiunii **make**. In acest moment modulul de kernel este pregătit pentru a fi încărcat. Dintre fisierele rezultate este cel cu extensia **ko**: **modul_kernel.ko**

Pentru a incarca modulul in kernel:

```
sudo insmod modul_kernel.ko
```

Pentru a vedea ca modulul s-a încărcat corect:

```
sudo dmesg
```

Se va afisa pe ecran mesajul pe care l-am dorit afisat la încărcarea modulului: „Incepere modul kernel”. Putem verifica daca modulul este inca incarcat in memorie cu ajutorul comenzii:

```
lsmod | grep "modul_kernel"
```

Pentru a scoate modulul din memorie:

```
sudo rmmod modul_kernel
```

Putem introduce si alt cod in modulul de kernel:

...

Tema

...

Bibliografie

Tim Bower. „System Calls“, http://faculty.salina.k-state.edu/tim/oss/Introduction/sys_calls.html#

Peter Jay Salzman, Michael Burian, Ori Pomerantz. „The Linux Kernel Module Programming Guide“, <http://www.tldp.org/LDP/lkmpg/2.6/html/lkmpg.html#AEN427>